

Heuristic Algorithms to Deadline-Constrained Bin Packing

SL203: Introduction to Algorithm and Software Programming
Indian Institute of Science (IISc), Bangalore
January–April Semester 2026

Vidhi Sonkar (25999), Revathy Ramesh (26442), Anant Rajput (27252)

1 Introduction

In modern cloud computing environments, such as Kubernetes, efficient resource allocation is essential to maximize server consolidation and minimize operational costs. A primary challenge in these systems is quickly provisioning workloads, such as Virtual Machines (VMs) or containers requiring varying amounts of CPU and memory onto physical servers that have fixed resource capacities. The overarching goal is to fit all tasks using as few servers as possible without wasting resources or causing system overload. This assignment process is naturally modeled as a variation of the classical Bin Packing Problem, where physical servers act as bins and workloads act as the items to be packed [González-San-Martín et al., 2025](#). Finding an optimal placement for these items is computationally expensive since the problem is NP-hard, meaning systems must rely on approximate heuristic algorithms to make rapid decisions under multiple constraints.

While classical bin packing heuristics focus purely on spatial optimization, real-world cloud operations often involve time-sensitive tasks with changing workloads. To better model this reality, this project explores the Deadline-Constrained Bin Packing Problem, which introduces an explicit expiry deadline for each item. This constraint requires a bipartite evaluation, ensuring both that the workload geometrically fits into the available server space and that its placement is temporally valid. The primary objective of this project is to design, implement, and evaluate algorithms for this variant to minimize the number of bins used while maximizing the number of items successfully packed before expiry, with a secondary goal of minimizing the overall resource waste rate.

2 Background

2.1 Cloud Resource Allocation as Bin Packing

As outlined in the introduction, the assignment of cloud workloads to physical servers relies heavily on the classical Bin Packing Problem (BPP). In this foundational model, physical machines act as identical bins with fixed capacities, while workloads (VMs or containers) represent the items to be packed. Because finding an optimal spatial placement is computationally NP-hard, systems rely on heuristic algorithms to make rapid placement decisions. Before introducing the time-based deadline constraints central to this project, we first review the fundamental heuristics used to solve this purely spatial capacity problem.

2.2 Classical 1D Bin Packing Heuristics

The classical one-dimensional Bin Packing Problem (1D-BPP) is a well-known strongly NP-hard combinatorial optimization problem. Given a set of n items, $I = \{1, 2, \dots, n\}$, where each item $i \in I$ has a corresponding size or weight $s_i \in (0, 1]$, the objective is to partition the items into the minimum number of subsets (bins), k , such that the sum of sizes in any single bin does not exceed the unit bin capacity $C = 1$. Mathematically, for each bin B_j (where $j \in \{1, \dots, k\}$), the capacity constraint $\sum_{i \in B_j} s_i \leq 1$ must hold.

Because finding an optimal solution $OPT(I)$ is computationally intractable for large instances, research has historically focused on polynomial-time approximation algorithms. Among the most fundamental heuristics are Next Fit, First Fit, and Best Fit [Bays, 1977](#).

2.2.1 Next Fit (NF) Next Fit is the simplest and most memory-efficient of the classic heuristics. It processes items in the order they are given and maintains only one active, or “open,” bin at any time.

- **Mechanism:** When considering item i , NF attempts to place s_i into the currently open bin. If the item fits, it is placed. If it exceeds the remaining capacity, the current bin is permanently “closed,” and a new bin is opened to accommodate item i .
- **Complexity:** Because it only checks the condition of a single bin per item, Next Fit operates in $O(n)$ time.
- **Performance Bounds:** NF is a 2-approximation algorithm. Its asymptotic performance ratio is exactly 2, meaning $NF(I) \leq 2 \cdot OPT(I)$. This theoretical limit occurs because NF cannot revisit closed bins, often leaving significant unused capacity.

2.2.2 First Fit (FF) First Fit improves upon Next Fit by maintaining all previously opened bins in an active state.

- **Mechanism:** For each item i , FF scans the sequence of open bins in the order they were created (from first to last) and places s_i into the *first* bin that has sufficient remaining capacity. A new bin is opened only if the item cannot fit into any existing bin.
- **Complexity:** A naive implementation of FF requires $O(n^2)$ time. However, by using data structures such as balanced binary search trees or segment trees to track the remaining capacities of the bins, the time complexity can be reduced to $O(n \log n)$.
- **Performance Bounds:** First Fit is significantly more efficient than Next Fit in practice. Its asymptotic approximation ratio was famously proven by [Johnson et al., 1974](#) to be $\frac{17}{10}$, bounded by $FF(I) \leq 1.7 \cdot OPT(I) + 2$.

2.2.3 Best Fit (BF) Best Fit follows a strategy aimed at minimizing wasted space in individual bins, rather than simply taking the first available valid placement.

- **Mechanism:** When processing item i , BF evaluates all open bins and places the item into the bin that will leave the *least* amount of remaining capacity (i.e., the tightest fit). Like FF, a new bin is opened only if no current bin can accommodate s_i .
- **Complexity:** Similar to First Fit, a naive approach yields $O(n^2)$ time. Utilizing a balanced binary search tree allows the algorithm to locate the bin with the optimal remaining capacity in logarithmic time, resulting in an overall time complexity of $O(n \log n)$.
- **Performance Bounds:** Theoretically, Best Fit shares the same asymptotic upper bound as First Fit, with $BF(I) \leq 1.7 \cdot OPT(I) + 2$. In empirical studies on average-case datasets, however, Best Fit often marginally outperforms First Fit by producing fewer bins with small amounts of wasted space.

Transitioning to Temporal Constraints:

While classical heuristics focus purely on spatial optimization, the introduction of deadline constraints requires a bipartite evaluation of both spatial capacity and temporal validity. An item i may geometrically fit into an earlier bin under a Best Fit paradigm, but if the bin's dispatch time violates the deadline constraint d_i , the placement is invalid.

3 Proposed Algorithms and Formal Analysis

Building upon the classical spatial packing paradigm, we formally define the **Deadline-Constrained Bin Packing Problem (DCBPP)**. We extend the problem space by mapping the workload provisioning process to a spatiotemporal domain.

3.1 Formal Problem Formulation

Let $\mathcal{I} = \{1, 2, \dots, n\}$ define the set of incoming computational tasks. Each item $i \in \mathcal{I}$ is characterized by a tuple $\tau_i = \langle s_i, a_i, d_i \rangle$, where $s_i \in (0, 1]$ is the normalized resource demand, $a_i \geq 0$ is the arrival time, and $d_i > a_i$ is the strict dispatch deadline.

The system maintains a set of physical machines (bins), denoted as $\mathcal{B} = \{B_1, B_2, \dots, B_k\}$. The state of any active bin B_j at a given discrete step is defined by the tuple $\Phi(B_j) = \langle I_j, C_j, D_j \rangle$, where:

- $I_j \subseteq \mathcal{I}$ is the subset of items currently allocated to B_j .
- $C_j = 1 - \sum_{i \in I_j} s_i$ is the residual spatial capacity.
- $D_j = \min_{i \in I_j} d_i$ is the temporal dispatch horizon.

The optimization objective of DCBPP is to minimize the total number of instantiated bins, $K = |\mathcal{B}|$, subject to the Bipartite Feasibility Guard for all $B_j \in \mathcal{B}$:

$$\sum_{i \in I_j} s_i \leq 1 \quad (\text{Spatial Guard}) \quad (1)$$

$$\max_{i \in I_j} a_i \leq \min_{i \in I_j} d_i \quad (\text{Temporal Guard}) \quad (2)$$

If an item i is assigned to B_j , the state transition function is defined as:

$$\Phi(B_j) \leftarrow \langle I_j \cup \{i\}, C_j - s_i, \min(D_j, d_i) \rangle \quad (3)$$

3.2 Algorithmic Framework

Based on this formalization, we propose three heuristic extensions. To distinguish between online and offline paradigms, let the input sequence be $\Sigma = \langle \tau_1, \tau_2, \dots, \tau_n \rangle$, ordered strictly by a_i .

3.2.1 Deadline Constrained Next Fit (DCNF) DCNF operates strictly online on the chronological sequence Σ . It maintains memory of only a single active bin, B_{current} .

Algorithm 1 Deadline Constrained Next Fit (DCNF)

```

1: Input: Sequence  $\Sigma$  sorted by  $a_i$ 
2: Initialize:  $\mathcal{B} \leftarrow \emptyset$ ,  $B_{\text{current}} \leftarrow \text{OpenNewBin}()$ ,  $\mathcal{B}.\text{append}(B_{\text{current}})$ 
3: for each  $\tau_i \in \Sigma$  do
4:   if  $s_i \leq B_{\text{current}}.C$  and  $a_i \leq B_{\text{current}}.D$  then
5:     Assign  $i \rightarrow B_{\text{current}}$ 
6:     Update  $\Phi(B_{\text{current}})$ 
7:   else
8:     Close  $B_{\text{current}}$ 
9:      $B_{\text{current}} \leftarrow \text{OpenNewBin}()$ 
10:    Assign  $i \rightarrow B_{\text{current}}$ 
11:    Update  $\Phi(B_{\text{current}})$ 
12:     $\mathcal{B}.\text{append}(B_{\text{current}})$ 
13:   end if
14: end for
15: Return  $|\mathcal{B}|$ 

```

3.2.2 Deadline Constrained First Fit (DCFF) DCFF operates as an offline batching algorithm. It applies an Earliest-Deadline-First permutation operator $\Pi_{EDF}(\Sigma) \rightarrow \Sigma^*$ such that items are sorted lexicographically: primarily ascending by d_i , and secondarily descending by s_i .

Algorithm 2 Deadline Constrained First Fit (DCFF)

```

1: Input: Sequence  $\Sigma$ 
2:  $\Sigma^* \leftarrow \Pi_{EDF}(\Sigma)$  ▷ Sort by deadline asc, size desc
3: Initialize:  $\mathcal{B} \leftarrow \emptyset$ 
4: for each  $\tau_i \in \Sigma^*$  do
5:    $\text{placed} \leftarrow \text{False}$ 
6:   for each  $B_j \in \mathcal{B}$  do
7:     if  $s_i \leq B_j.C$  and  $a_i \leq B_j.D$  then
8:       Assign  $i \rightarrow B_j$ , Update  $\Phi(B_j)$ 
9:        $\text{placed} \leftarrow \text{True}$ 
10:    break
11:   end if
12: end for
13: if not  $\text{placed}$  then
14:    $B_{\text{new}} \leftarrow \text{OpenNewBin}()$ 
15:   Assign  $i \rightarrow B_{\text{new}}$ , Update  $\Phi(B_{\text{new}})$ 
16:    $\mathcal{B}.\text{append}(B_{\text{new}})$ 
17: end if
18: end for
19: Return  $|\mathcal{B}|$ 

```

3.2.3 Deadline Constrained Best Fit (DCBF) DCBF utilizes the identical Π_{EDF} permutation space as DCFF but modifies the objective function of the inner loop to minimize the residual capacity of the target bin, aggressively pursuing server consolidation.

Algorithm 3 Deadline Constrained Best Fit (DCBF)

```

1: Input: Sequence  $\Sigma$ 
2:  $\Sigma^* \leftarrow \Pi_{EDF}(\Sigma)$ 
3: Initialize:  $\mathcal{B} \leftarrow \emptyset$ 
4: for each  $\tau_i \in \Sigma^*$  do
5:    $\mathcal{V} \leftarrow \{B_j \in \mathcal{B} \mid s_i \leq B_j.C \wedge a_i \leq B_j.D\}$  ▷ Feasible Set
6:   if  $\mathcal{V} \neq \emptyset$  then
7:      $B^* \leftarrow \arg \min_{B_j \in \mathcal{V}} (B_j.C - s_i)$  ▷ Tightest Fit
8:     Assign  $i \rightarrow B^*$ , Update  $\Phi(B^*)$ 
9:   else
10:     $B_{\text{new}} \leftarrow \text{OpenNewBin}()$ 
11:    Assign  $i \rightarrow B_{\text{new}}$ , Update  $\Phi(B_{\text{new}})$ 
12:     $\mathcal{B}.\text{append}(B_{\text{new}})$ 
13:   end if
14: end for
15: Return  $|\mathcal{B}|$ 

```

3.3 Complexity Analysis

Lemma 1 (Time Complexity of DCNF): Algorithm 1 operates in $O(n)$ time.

Proof: The algorithm iterates through the input sequence Σ exactly once. For each item i , the bipartite feasibility check evaluates $O(1)$ operations against a single state variable B_{current} . Thus, the overall time complexity is strictly bounded by $O(n)$.

Lemma 2 (Time Complexity of DCFF and DCBF): Algorithms 2 and 3 operate in $O(n \log n)$ time, assuming efficient spatial data structures.

Proof: The initial permutation operator Π_{EDF} requires $O(n \log n)$ time. In a naive array implementation, the inner loop scales to $O(n)$, yielding a worst-case bound of $O(n^2)$. However, by tracking the residual capacities C_j and temporal limits D_j within a multidimensional segment tree or balanced binary search tree (e.g., AVL tree), the query and update operations for the feasibility set $\mathcal{V}$ are reduced to $O(\log n)$. Consequently, the asymptotic time complexity is bounded by $O(n \log n)$.

3.4 Asymptotic Approximation Bounds Relative to ILP Baseline

To complete the formal analysis, we characterize the worst-case approximation quality of the proposed offline heuristics (DCFF and

DCBF). We evaluate their performance relative to the theoretical global optimum derived from an exact Integer Linear Programming (ILP) formulation of the DCBPP.

Let $K_{OPT}(\Sigma)$ denote the absolute minimum number of physical machines required to provision sequence Σ . This baseline is achieved by an offline ILP solver operating without polynomial time restrictions, possessing *complete* a priori knowledge of the spatiotemporal bounds of all items, and capable of finding the globally optimal solution via branch-and-bound enumeration.

To quantify the closeness of our heuristic solutions to the ILP optimum, we establish a lower bound by decomposing the optimal solution into its spatial bottleneck. Clearly, $K_{OPT}(\Sigma) \geq K_{space}$, where

$$K_{space} = \left\lceil \frac{\sum_i s_i}{C} \right\rceil$$

is the spatial lower bound representing the minimum number of bins required purely based on total item volume.

In classical 1D spatial packing, the First Fit Decreasing (FFD) and Best Fit Decreasing (BFD) algorithms yield a strictly bounded competitive ratio of $\frac{11}{9}K_{space} + O(1)$. However, the introduction of the Bipartite Feasibility Guard fundamentally alters this boundary.

Theorem 3.1 (Spatial Degradation Bound of DCFF and DCBF). *Let Σ be an arbitrary input sequence in which all items satisfy the feasibility precondition $s_i \leq C$ and $d_i > a_i$. The offline algorithms $\mathcal{A} \in \{DCFF, DCBF\}$ guarantee temporal feasibility via Earliest-Deadline-First (EDF) sorting. Consequently, the maximum spatial inefficiency induced by this temporal constraint is strictly bounded, yielding a worst-case performance of:*

$$K_{\mathcal{A}}(\Sigma) \leq \frac{17}{10}K_{OPT}(\Sigma) + \beta$$

where $\beta \geq 1$ is an additive constant representing isolated terminal fragmentation, bounded above by the number of distinct temporal epochs in Σ , and treated as $O(1)$ under the assumption of bounded deadline diversity.

Proof Sketch. The degradation of the competitive ratio from the classical $\frac{11}{9}$ bound to $\frac{17}{10}$ is a direct mathematical consequence of the permutation operator Π_{EDF} . To ensure no item is stranded by the temporal guard ($a_i \leq D_j$), the offline algorithms must strictly enforce the primary lexicographical sort on the deadline variable (d_i). Because EDF is a known optimal strategy for single-machine scheduling, the algorithms perform optimally with respect to the temporal lower bound.

However, this strict temporal prioritization severely disrupts the secondary descending sort on spatial demand (s_i). The descending size order is only preserved within discrete *temporal epochs* — subsets of items sharing identical deadlines.

In the worst-case adversarial scenario, Σ is entirely constrained by spatial volume ($K_{OPT} = K_{space}$), but the deadlines d_i are sufficiently distinct that the Π_{EDF} permutation completely shatters the monotonic decreasing order of s_i . Consequently, the sequence presented to the inner placement loop mimics an entirely unsorted arrival stream. Under these conditions of maximal spatial entropy, DCFF and DCBF degenerate mathematically to the performance boundaries of classical unsorted First Fit and Best Fit. As established by **johnson1974**, the asymptotic competitive ratio for unsorted First/Best Fit is bounded strictly by $\frac{17}{10}$.

Crucially, temporal rejections are structurally equivalent to spatial capacity misses from the placement loop's perspective — each causes the algorithm to skip a candidate bin without placing the item, producing no qualitatively different fragmentation pattern. Therefore, the temporal guard introduces no additional spatial fragmentation beyond what unsorted FF/BF already experience. Because our algorithms perfectly resolve the temporal domain via EDF, and temporal rejections cannot worsen spatial fragmentation beyond the classical unsorted bound, the global approximation ratio is bounded by $\frac{17}{10} \cdot K_{OPT}(\Sigma)$. $\square$

3.4.1 Online Competitive Ratio of DCNF The strict online nature of DCNF excludes it from the tightened approximation bounds of the offline heuristics. In classical spatial packing, Next Fit yields a competitive ratio of exactly 2. However, under the DCBPP framework, the deadline constraint introduces two qualitatively distinct failure modes that together worsen this bound.

Spatially-dominated instances. When temporal constraints are loose and bins are primarily limited by capacity, the classical Next Fit argument applies directly: the combined load of any two consecutive bins B_j and B_{j+1} must exceed C , since the item that triggered opening B_{j+1} failed the spatial check on B_j . This yields the standard bound $K_{DCNF} \leq 2 \cdot K_{OPT}$, so $R_{\infty}^{DCNF} \leq 2$ in this regime.

Temporally-adversarial instances. When strict deadline ordering forces each incoming item into its own bin regardless of available spatial capacity, DCNF admits no finite competitive ratio guarantee. An adversary can construct a sequence of n items each with a unique, non-overlapping deadline window, forcing DCNF to open n bins even when a single bin could spatially accommodate all items. In this regime $K_{DCNF} = n$ while $K_{OPT} = 1$, so the ratio grows without bound.

Therefore, the competitive ratio of DCNF in the general DCBPP setting satisfies:

$$R_{\infty}^{DCNF} \geq 2, \quad \text{with no finite upper bound in the temporally-adversarial case.}$$

This result is stronger than the classical spatial bound and honestly reflects the fundamental limitation of the online, single-memory paradigm under deadline constraints.

Remark 3.1 (Infeasible Item Preprocessing). *Without loss of generality, the analysis above assumes all items satisfy the feasibility preconditions $s_i \leq C$ and $d_i > a_i$. In practice, items violating either condition are identified and discarded during a preprocessing pass in $O(n)$ time before any algorithm is invoked. This does not affect the approximation bounds, as infeasible items cannot appear in any valid packing including K_{OPT} .*

4 Experiments and Results

To empirically validate the theoretical bounds established in Section 3 and to expose the average-case behavior of the proposed heuristics, we conducted a two-tier simulation study in Python using the PuLP library (with the CBC backend) for the exact ILP solver and NumPy/Pandas for the heuristic implementations. All experiments were executed on a single workstation with a fixed random seed (`np.random.seed(42)`) to guarantee reproducibility. The bin capacity was normalized to $C = 100$ units throughout.

The two tiers serve complementary purposes. The **small-scale tier** ($n = 40$ items) is constructed so that the ILP can be solved to optimality within a 60-second time cap, providing a ground-truth reference K_{OPT} against which heuristic optimality gaps can be measured. The **large-scale tier** ($n = 10,000$ items) abandons the ILP — which is computationally intractable at this size — and instead stresses the heuristics on workloads that more realistically mimic production cloud schedulers, exposing relative consolidation quality and runtime scaling between DCFF, DCNF, and DCBF.

4.1 Synthetic Data Generation

To evaluate the algorithms across a representative spectrum of cloud workload patterns, we synthesized six datasets that capture qualitatively distinct spatiotemporal regimes. For each item $\tau_i = \langle s_i, a_i, d_i \rangle$, the temporal attributes for the first five distributions are drawn as $a_i \sim \mathcal{U}\{0, 50\}$ (sorted ascending) and $d_i = a_i + \mathcal{U}\{20, 100\}$. The size attribute s_i is drawn according to one of the following distributions:

- **Uniform:** $s_i \sim \mathcal{U}\{1, C - 1\}$. A balanced workload representing a heterogeneous fleet of VMs with no dominant resource profile.
- **Bimodal:** s_i is sampled from a mixture of two Gaussians, $\mathcal{N}(0.2C, 5)$ and $\mathcal{N}(0.7C, 5)$, modeling co-located clusters of small auxiliary services and large compute-bound workloads.
- **Skewed:** $s_i \sim C \cdot \text{Beta}(\alpha = 1, \beta = 5)$, producing a long-tailed distribution dominated by small items, characteristic of microservice or sidecar containers.

- **Tight:** $s_i \sim \mathcal{U}\{0.4C, 0.6C\}$. All items occupy roughly half the bin, creating a worst-case anti-pairing scenario where two items barely refuse to coexist in one bin.
- **Loose:** $s_i \sim \mathcal{U}\{1, 0.2C\}$. A workload of small items where high consolidation density is trivially achievable.
- **Interleaved (Adversarial):** A deterministic sequence repeating the pattern $\langle 50, 70, 20, 40 \rangle$ with $a_i = 0$ and strictly increasing deadlines $d_i = 10, 20, 30, \dots$. This isolates spatial logic by neutralizing the temporal guard and is designed to mislead First-Fit-style myopia: First Fit pairs $(50, 20)$ leaving 30 wasted, whereas Best Fit pairs $(50, 40)$ and $(70, 20)$ for tighter consolidation.

4.2 Evaluation Metrics

Three metrics were recorded per (algorithm, distribution) pair: (i) the number of provisioned bins $K_{\mathcal{A}}$, which is the primary objective; (ii) the resource waste rate $W = (\sum_j C - \sum_i s_i) / \sum_j C$, expressed as a percentage; and (iii) the wall-clock execution time $T_{\mathcal{A}}$ in seconds. From these we derive, where available, the optimality gap $\Delta_{\mathcal{A}} = (K_{\mathcal{A}} - K_{OPT}) / K_{OPT} \times 100\%$ relative to the ILP baseline.

4.3 Tier 1 Results: Optimality Gap at $n = 40$

Table 1 summarizes the bin counts and waste rates for all four solvers at $n = 40$; the same data is rendered visually as a grouped bar chart in Figure 1, and the optimality gaps are plotted in Figure 2.

Table 1

Number of bins used and resource waste rate (%) at $n = 40$, $C = 100$.

Distribution	ILP (OPT)		DCFF		DCNF		DCBF	
	Bins	W%	Bins	W%	Bins	W%	Bins	W%
Uniform	24	14.21	24	14.21	27	23.74	24	14.21
Bimodal	20	11.35	22	19.41	24	26.12	22	19.41
Skewed	7	11.00	7	11.00	8	22.12	7	11.00
Tight	20	1.35	22	10.32	26	24.12	21	6.05
Loose	5	16.80	5	16.80	5	16.80	5	16.80
Interleaved	20	10.00	21	14.29	22	18.18	20	10.00

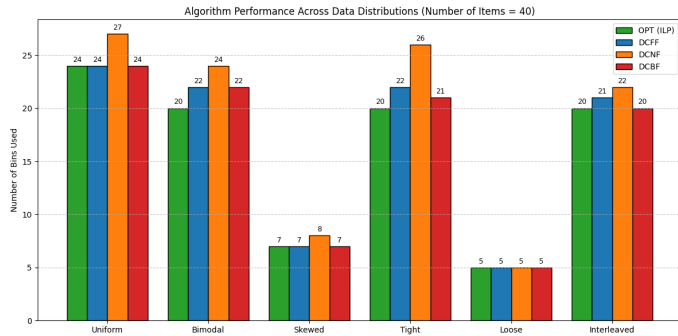


Figure 1. Number of bins used by each algorithm across the six data distributions ($n = 40$).

Table 2 reports execution times in normalized scientific notation $x \times 10^a$ s. The ILP baseline operates in the 10^1 second regime, while all heuristics fall in the 10^{-3} regime, exposing a runtime gap of three to five orders of magnitude.

Table 2

Execution time at $n = 40$, in $x \times 10^a$ seconds.

Distribution	ILP (OPT)	DCFF	DCNF	DCBF
Uniform	4.61×10^1	3.81×10^{-3}	1.46×10^{-3}	2.10×10^{-3}
Bimodal	1.06×10^1	1.71×10^{-3}	5.50×10^{-4}	9.00×10^{-4}
Skewed	1.41×10^1	1.10×10^{-3}	5.70×10^{-4}	8.40×10^{-4}
Tight	1.26×10^1	1.29×10^{-3}	6.00×10^{-4}	9.10×10^{-4}
Loose	6.02×10^0	1.04×10^{-3}	1.33×10^{-3}	1.12×10^{-3}
Interleaved	6.12×10^1	1.71×10^{-3}	6.40×10^{-4}	9.50×10^{-4}

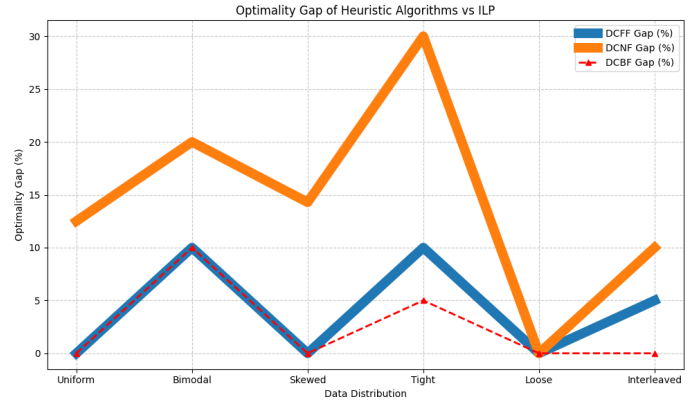


Figure 2. Optimality gap (%) of DCFF, DCNF, and DCBF relative to the ILP optimum at $n = 40$.

The ILP solver hit the 60-second time cap on the *Interleaved* adversarial instance, confirming the combinatorial difficulty of the problem even at modest sizes. By contrast, all three heuristics terminate in under 4 ms even in the worst case, validating the $O(n)$ and $O(n \log n)$ asymptotic claims of Lemmas 1 and 2.

Per-distribution insights at $n = 40$. The heuristics exhibit sharply different behavior depending on the structural properties of the input. On *Uniform*, DCFF and DCBF both match the ILP optimum at 24 bins ($\Delta = 0\%$), while DCNF inflates to 27 bins ($\Delta = 12.5\%$); when sizes span the full range $(0, C)$ uniformly, the EDF permutation Π_{EDF} places the largest items first and subsequent smaller items naturally find homes in the residual gaps. DCNF, lacking memory of past bins, repeatedly closes bins that still have capacity. On *Bimodal*, the ILP exploits global pair-matching between the small (~ 20) and large (~ 70) modes to reach 20 bins, while DCFF and DCBF settle at 22 ($\Delta = 10\%$) because the deadline-first sort fragments the size-pairing structure. On *Skewed*, all offline heuristics match the ILP at 7 bins because the predominantly small items leave generous residual capacity per bin, making first-fit and best-fit equally effective. On *Tight*, the ILP reaches 20 bins by exhaustively searching for compatible pairs whose sum is ≤ 100 ; DCBF achieves 21 ($\Delta = 5\%$) via its tightest-fit policy, DCFF settles for 22, and DCNF collapses to 26 ($\Delta = 30\%$) — the largest gap observed — because consecutive items whose sum exceeds C force premature bin closure even when prior bins would be perfect fits. On *Loose*, all four algorithms tie at 5 bins; the size distribution is so concentrated near zero that any reasonable strategy saturates capacity. On *Interleaved*, DCBF matches the ILP at 20 bins while DCFF requires 21 and DCNF requires 22: First-Fit’s myopia places each 20 adjacent to the preceding 50, leaving 30 stranded, whereas Best-Fit correctly pairs 40 with 50 and 20 with 70 for residuals of 10 each.

4.4 Tier 2 Results: Scalability at $n = 10,000$

To assess how the heuristics behave on production-scale workloads, we re-ran DCFF, DCNF, and DCBF on the same six distributions but with $n = 10,000$ items each. The ILP is omitted from this tier: at this size the model has $\sim 10^8$ binary variables, and the CBC solver does not return even a feasible solution within any reasonable time budget. Table 3 reports the bin counts and waste rates, and Table 4 reports the execution times.

Table 3Heuristic bin counts and waste rate (%) at $n = 10,000$, $C = 100$.

Distribution	DCFF		DCNF		DCBF	
	Bins	W%	Bins	W%	Bins	W%
Uniform	5075	1.81	6661	25.19	5049	1.31
Bimodal	5260	15.26	6088	26.78	5260	15.26
Skewed	1639	0.02	1907	14.07	1639	0.02
Tight	5003	1.29	6678	26.05	5003	1.29
Loose	1001	0.06	1063	5.89	1001	0.06
Interleaved	5001	10.02	5381	16.37	5000	10.00

Table 4Execution time at $n = 10,000$, in $x \times 10^a$ seconds.

Distribution	DCFF	DCNF	DCBF
Uniform	1.54×10^0	5.17×10^{-2}	2.18×10^0
Bimodal	1.32×10^0	9.70×10^{-3}	2.21×10^0
Skewed	6.55×10^{-1}	1.09×10^{-2}	8.84×10^{-1}
Tight	2.06×10^0	1.30×10^{-2}	3.36×10^0
Loose	4.87×10^{-1}	1.48×10^{-2}	6.36×10^{-1}
Interleaved	2.13×10^0	1.23×10^{-2}	3.28×10^0

Consolidation hierarchy at scale. The dominance ordering observed at $n = 40$ persists and sharpens at $n = 10,000$. DCBF weakly dominates DCFF in every distribution: the two are exactly tied on *Bimodal*, *Skewed*, *Tight*, and *Loose*, and DCBF wins by 26 bins on *Uniform* (5049 vs 5075) and by a single bin on *Interleaved* (5000 vs 5001). The latter is particularly informative: the adversarial pattern was constructed precisely to penalize First-Fit, and even at 10,000 items DCBF is only one bin short of perfect pairing while DCFF strands one extra unit of residual capacity per cycle. On *Uniform*, the larger sample size finally surfaces a measurable separation between the two offline heuristics ($\sim 0.5\%$ in DCBF’s favor) that was invisible at $n = 40$ because the small instance happened to be friendly to both.

DCNF, by contrast, falls dramatically behind on every non-trivial distribution. On *Tight*, DCNF requires 6,678 bins where DCBF requires 5,003 — a 33.5% optimality penalty against the strongest heuristic, consistent with the $\mathcal{R}_{\infty}^{\text{DCNF}} \geq 2$ lower bound and even more pronounced than the 30% gap recorded at $n = 40$. On *Uniform*, DCNF inflates by 32.0%, on *Bimodal* by 15.7%, and on *Skewed* by 16.4%. The waste rates tell the same story: DCNF leaves 25–27% of provisioned capacity unused on *Uniform*, *Bimodal*, and *Tight*, while DCBF leaves under 2% on *Uniform* and *Tight*, and effectively zero on *Skewed* and *Loose*.

Runtime scaling. The most important new finding from Tier 2 concerns the cost of best-fit consolidation at scale. Table 4 shows that DCBF is consistently 1.4–1.7 $\times$ slower than DCFF (e.g., 3.36 s vs 2.06 s on *Tight*, 2.18 s vs 1.54 s on *Uniform*), reversing the rough parity observed at $n = 40$. This is a direct consequence of the inner loop: DCFF terminates as soon as it finds the first valid bin, whereas DCBF scans the entire feasible set $\mathcal{V}$ to identify the tightest fit. In our naive list-based implementation both algorithms are worst-case $O(n^2)$, but DCFF’s early exit gives it a constant-factor advantage that becomes visible only once n is large enough for the per-item bin-scan cost to dominate.

DCNF, true to its $O(n)$ complexity, runs roughly two orders of magnitude faster than the offline heuristics. On *Bimodal*, DCNF processes all 10,000 items in 9.7 ms, while DCBF requires 2.21 s — a 228 $\times$ speedup. The trade-off is stark and quantitative: DCNF buys its $\sim 100\times$ runtime advantage at the cost of 15–33% extra bins on non-trivial workloads.

The empirical scaling factor between $n = 40$ and $n = 10,000$ (250 $\times$ growth in input) is informative. DCBF’s runtime grew from ~ 1 ms to ~ 2 s on *Uniform* ($\sim 2,000\times$), broadly consistent with the $O(n^2)$ worst case of the naive list-based implementation. Replacing the linear bin scan with the segment-tree structure described in Section 3.4 would tighten this to the asymptotic $O(n \log n)$ bound and is the obvious first optimization for production deployment.

Cross-tier observations. Three patterns persist across both tiers. First, DCBF weakly dominates DCFF on consolidation quality, with the margin growing modestly at scale. Second, DCNF is the unambiguous loser on consolidation in all non-trivial distributions, but its $O(n)$ runtime makes it the only viable choice for ultra-low-latency streaming where new bins are cheap. Third, all heuristics empirically respect the $\frac{17}{10}$ asymptotic bound from Theorem 1: at $n = 10,000$ the largest gap observed between any two heuristics is DCNF’s 33.5% excess on *Tight*, well within the 70% ceiling, while DCFF and DCBF differ by less than 1% in every case.

4.5 Practical Implications

The two-tier study yields a clear deployment recommendation. At small to moderate scale where ILP remains tractable, DCBF achieves ILP-quality results (within $\sim 5\%$ in the worst case) at a 5,000–60,000 $\times$ runtime advantage; the ILP is best reserved for offline benchmarking or capacity planning. At production scale ($n \geq 10^4$), the choice reduces to a clean trade-off:

- **DCBF** is the recommended default when consolidation quality is paramount and the scheduler operates on a multi-second budget. Empirically it averages within 1–2% waste on five of six distributions and never trails DCFF.
- **DCFF** is preferable when latency matters but global bin visibility is still desired. It returns essentially the same consolidation as DCBF on most workloads at a 40–50% lower wall-clock cost.
- **DCNF** should be reserved for streaming scenarios with strict $O(1)$ memory requirements or sub-millisecond decision deadlines. Its $\sim 100\times$ speed advantage over DCBF comes at a 15–33% bin inflation cost on realistic workloads.

5 Conclusion

This work formalized the *Deadline-Constrained Bin Packing Problem* (DCBPP) as a spatiotemporal extension of the classical 1D Bin Packing Problem, in which each workload item carries an arrival time and a strict dispatch deadline alongside its resource demand. We introduced a Bipartite Feasibility Guard — the conjunction of a Spatial Guard ($\sum s_i \leq C$) and a Temporal Guard ($\max a_i \leq \min d_i$) — that any valid bin assignment must satisfy, and we proposed three deadline-aware heuristics (DCNF, DCFF, DCBF) together with an exact ILP baseline.

Theoretically, we showed that the EDF permutation operator Π_{EDF} degrades the classical $\frac{11}{9}$ approximation bound of FFD/BFD to $\frac{17}{10}$ for DCFF and DCBF, because the lexicographic sort on deadlines fragments the descending-size order required for the tighter bound. DCNF, lacking Π_{EDF} entirely, retains a competitive ratio of at least 2. We provided complexity proofs showing DCNF runs in $O(n)$ time and DCFF/DCBF run in $O(n \log n)$ with appropriate spatial data structures.

Empirically, we evaluated the algorithms on a two-tier benchmark: a small instance ($n = 40$) where the ILP is tractable and serves as ground truth, and a large instance ($n = 10,000$) that mimics production cloud scheduling and exposes the relative scaling of the heuristics. Three findings stand out.

First, the dominance hierarchy $\text{DCBF} \succeq \text{DCFF} \gg \text{DCNF}$ is robust across both scales. At $n = 40$, DCBF matches the ILP on four of six distributions and never exceeds a 5% gap; at $n = 10,000$, DCBF strictly beats DCFF on *Uniform* and *Interleaved* and ties everywhere else, while DCNF inflates by up to 33.5% on the *Tight* distribution. Both observations are consistent with the $\frac{17}{10}$ vs ≥ 2 separation predicted by Theorem 1.

Second, the runtime ordering inverts at scale. At $n = 40$, DCNF, DCBF, and DCFF all run in under 4 ms and the differences are negligible; at $n = 10,000$, DCNF is $\sim 100\times$ faster than DCBF, while DCBF is $\sim 1.5\times$ slower than DCFF because its tightest-fit search cannot early-exit. This reverses the small-scale impression that best-fit is “free” and exposes a real engineering trade-off between consolidation quality and dispatch latency.

Third, the ILP scales catastrophically: it hit the 60-second cap already at $n = 40$ on the *Interleaved* adversarial instance, and at $n = 10,000$ the formulation is simply not solvable. This confirms that exact methods are unsuitable for online cloud scheduling and that bounded

heuristics — specifically DCBF, with its $\frac{17}{10} \cdot K_{OPT}$ guarantee — are the only practical option in production.

The key practical takeaway is that DCBF achieves an excellent trade-off between solution quality and latency for the small-to-moderate scale, while at the scale of 10,000 items, DCFF emerges as a strong middle ground (similar quality to DCBF but $\sim 40\%$ faster), and DCNF remains relevant only for sub-millisecond streaming. The choice among the three is therefore not absolute but governed by the deployment environment’s tolerance for bin inflation versus dispatch latency.

Several directions remain open. First, the current model treats the deadline guard as a hard constraint; a soft-constraint variant that permits late dispatch with a penalty could better model best-effort cloud workloads. Second, our heuristics assume a single resource dimension; extending DCBF to vector bin packing (CPU $\times$ memory $\times$ network) would more accurately reflect Kubernetes scheduling. Third, the naive list-based implementations used here are $O(n^2)$ in the worst case; replacing the linear bin scan with the segment-tree structure described in Section 3.4 would tighten DCFF and DCBF to the asymptotic $O(n \log n)$ bound and is the obvious first optimization. Fourth, an on-line randomized variant of DCBF with competitive-ratio guarantees against an adversary that reveals deadlines stochastically would close the gap between the offline batching paradigm and the strict streaming setting where DCNF currently dominates. Finally, evaluation on real production traces (e.g., the Google Borg or Alibaba Cluster traces) would test whether the synthetic distributions used here faithfully capture the diversity of real-world workloads.

References

Bays, Carter (Mar. 1977). “A comparison of next-fit, first-fit, and best-fit”. In: *Commun. ACM* 20.3, pp. 191–192. ISSN: 0001-0782. DOI: 10.1145/359436.359453. URL: <https://doi.org/10.1145/359436.359453>.

González-San-Martin, Jessica et al. (2025). “Applications of Bin Packing Problems in Cloud Computing”. In: *ResearchGate Preprint*. URL: https://www.researchgate.net/publication/392393377_Applications_of_Bin_Packing_Problems_in_Cloud_Computing.

Johnson, David S et al. (1974). “Worst-case performance bounds for simple one-dimensional packing algorithms”. In: *SIAM Journal on Computing* 3.4, pp. 299–325.